Phala Network E2E 테스트 시스템 분석

1. E2E 테스트 개요

Phala Network의 E2E(End-to-End) 테스트 시스템은 **전체 스택의 통합 테스트**를 수행하여 블록체인 노드, pRuntime, Pherry 릴레이어, 그리고 스마트 컨트랙트가 함께 올바르게 작동하는지 검증합니다.

2. 테스트 환경 구성

2.1 패키지 의존성

```
{
"dependencies": {
// Phala SDK
"@phala/sdk": "file:../frontend/packages/sdk",
// Polkadot 생태계
"@polkadot/api": "Substrate API 클라이언트",
"@polkadot/api-contract": "스마트 컨트랙트 API",
"@polkadot/util-crypto": "암호화 유틸리티",
// 테스트 프레임워크
"chai": "어서션 라이브러리",
"mocha": "테스트 러너",
// 유틸리티
```

```
"axios": "HTTP 클라이언트",
"bn.js": "큰 수 연산",
"dotenv": "환경 변수 관리",
"portfinder": "사용 가능한 포트 찾기"
}
}
```

2.2 테스트 설정

```
// 테스트 환경 설정 (fullstack.js 기반)
require('dotenv').config();

// 바이너리 경로 설정
const pathNode = path.resolve('../target/release/phala-node');
const pathRelayer = path.resolve('../target/release/pherry');
const pathPRuntime = path.resolve('../standalone/pruntime/bin/pruntime');

// 실행 모드 설정
const inSgx = process.env.E2E_IN_SGX == '1';
const quickMode = process.env.QUICK == '1';
const sgxLoader = "gramine-sgx";

// 네트워크 파라미터
```

```
const CENTS = 10_000_000_000;
const blockInterval = quickMode ? 100 : 300; // ms
const gossipDuration = 20; // ms

console.log(`Testing in ${inSgx ? "SGX Hardware" : "Software"} mode`);
```

3. 테스트 아키텍처

3.1 프로세스 관리 시스템

```
// 프로세스 관리 유틸리티 (추정)
class Process {
constructor(command, args, options = {}) {
this.command = command;
this.args = args;
this.options = options;
this.process = null;
this.logs = [];
}
async start() {
return new Promise((resolve, reject) => {
this.process = spawn(this.command, this.args, {
stdio: ['pipe', 'pipe', 'pipe'],
```

```
...this.options
});
// 로그 수집
this.process.stdout.on('data', (data) => {
const log = data.toString();
this.logs.push(log);
if (this.options.verbose) {
console.log(`[${this.command}] ${log}`);
}
});
this.process.stderr.on('data', (data) => {
const log = data.toString();
this.logs.push(log);
if (this.options.verbose) {
console.error(`[${this.command}] ${log}`);
}
});
// 프로세스 시작 확인
this.process.on('spawn', () => {
resolve(this.process.pid);
});
this.process.on('error', reject);
});
```

```
}
async stop() {
if (this.process) {
this.process.kill('SIGTERM');
// 강제 종료 타임아웃
setTimeout(() => {
if (this.process && !this.process.killed) {
this.process.kill('SIGKILL');
}
}, 5000);
}
}
getLogs() {
return this.logs.join('\n');
}
}

// 임시 디렉토리 관리
class TempDir {
constructor(prefix = 'phala-test-') {
this.path = fs.mkdtempSync(path.join(os.tmpdir(), prefix));
}
cleanup() {
```

```
fs.rmSync(this.path, { recursive: true, force: true });
}
getPath(subPath = '') {
return path.join(this.path, subPath);
}
}
```

3.2 pRuntime API 클라이언트

```
// pRuntime API 클라이언트 (추정)
class PRuntimeApi {
constructor(endpoint) {
this.endpoint = endpoint;
this.client = axios.create({
baseURL: endpoint,
timeout: 30000,
headers: {
'Content-Type': 'application/json'
}
});
}
async getInfo() {
const response = await this.client.get('/get_info');
```

```
return response.data;
}
async syncHeader(headerData) {
const response = await this.client.post('/sync_header', {
header_data: headerData
});
return response.data;
}
async syncParaHeader(headerData) {
const response = await this.client.post('/sync_para_header', {
header_data: headerData
});
return response.data;
}
async dispatchBlocks(blocks) {
const response = await this.client.post('/dispatch_blocks', {
blocks: blocks
});
return response.data;
}
async query(contractId, inputData, origin = null) {
const response = await this.client.post('/query', {
contract_id: contractId,
```

```
input_data: inputData,
origin: origin
});
return response.data;
}
async getContractInfo(contractId) {
const response = await this.client.get(`/get_contract_info/${contractId}`);
return response.data;
}
async getWorkerState() {
const response = await this.client.get('/get_worker_state');
return response.data;
}
}
```

4. 테스트 시나리오

4.1 풀스택 테스트 구조

```
// 메인 테스트 스위트 (추정)
describe('A full stack', function () {
let nodeProcess;
let pRuntimeProcess;
```

```
let relayerProcess;
let api;
let keyring;
let alice, bob, charlie;
let tempDir;
// 테스트 타임아웃 설정
this.timeout(120000);
before(async function () {
// 1. 환경 준비
await setupTestEnvironment();
// 2. 프로세스 시작
await startAllProcesses();
// 3. API 연결
await connectToBlockchain();
// 4. 계정 설정
setupAccounts();
});
after(async function () {
// 정리 작업
await cleanupTestEnvironment();
});
describe('Blockchain Node', function () {
it('should start successfully', async function () {
```

```
assert.isTrue(nodeProcess.isRunning());
});
it('should produce blocks', async function () {
const initialBlock = await api.rpc.chain.getBlock();
await sleep(blockInterval * 3);
const laterBlock = await api.rpc.chain.getBlock();
assert.isTrue(
laterBlock.block.header.number.toNumber() >
initialBlock.block.header.number.toNumber()
);
});
});
describe('pRuntime', function () {
it('should start and be ready', async function () {
const info = await pRuntimeApi.getInfo();
assert.equal(info.state, 'Ready');
});
it('should sync with blockchain', async function () {
// 헤더 동기화 테스트
const header = await api.rpc.chain.getHeader();
const result = await pRuntimeApi.syncHeader(header.toHex());
assert.isTrue(result.success);
});
```

```
});

describe('Pherry Relayer', function () {

it('should relay blocks to pRuntime', async  function () {

// 릴레이어가 블록을 pRuntime에 전달하는지 확인

const  initialInfo  =  await  pRuntimeApi.getInfo();

await  sleep(blockInterval * 5);

const  laterInfo  =  await  pRuntimeApi.getInfo();

assert.isTrue(

laterInfo.blocknum > initialInfo.blocknum

);

});

});

});
```

4.2 워커 등록 테스트

```
// 워커 등록 테스트 (추정)

describe('Worker Registration', function () {

let  workerInfo;

it('should register a worker', async  function () {

// 1. 워커 정보 생성

const  workerPubkey  =  keyring.addFromUri('//Worker').publicKey;

const  attestation  =  await  generateMockAttestation(workerPubkey);
```

```
// 2. 워커 등록 트랜잭션
const tx = api.tx.phalaRegistry.registerWorker(
workerPubkey,
attestation.payload,
attestation.signature
);
await new Promise((resolve, reject) => {
tx.signAndSend(alice, ({ status, events }) => {
if (status.isInBlock) {
// 이벤트 확인
const workerRegisteredEvent = events.find(
({ event }) => api.events.phalaRegistry.WorkerRegistered.is(event)
);
assert.isDefined(workerRegisteredEvent);
workerInfo = workerRegisteredEvent.event.data;
resolve();
} else if (status.isInvalid) {
reject(new Error('Transaction invalid'));
}
});
});
});
it('should query worker info', async function () {
```

```
const onChainWorkerInfo = await api.query.phalaRegistry.workers(workerInfo.w
assert.isTrue(onChainWorkerInfo.isSome);
const worker = onChainWorkerInfo.unwrap();
assert.equal(worker.state.toString(), 'Ready');
});
});
```

4.3 스마트 컨트랙트 테스트

```
// 스마트 컨트랙트 테스트 (추정)
describe('Smart Contract Deployment and Execution', function () {
let contractId;
let contract;
it('should deploy a contract', async function () {
// 1. 컨트랙트 코드 로드
const contractCode = fs.readFileSync('./contracts/flipper.wasm');
const contractAbi = JSON.parse(fs.readFileSync('./contracts/flipper.json'));
// 2. 클러스터 생성
const createClusterTx = api.tx.phalaComputation.createCluster();
const clusterId = await new Promise((resolve, reject) => {
createClusterTx.signAndSend(alice, ({ status, events }) => {
if (status.isInBlock) {
const clusterCreatedEvent = events.find(
```

```
({ event }) => api.events.phalaComputation.ClusterCreated.is(event)
);
resolve(clusterCreatedEvent.event.data.clusterId);
}
});
});
// 3. 컨트랙트 배포
const deployTx = api.tx.phalaComputation.deployContract(
clusterId,
contractCode,
'0x', // constructor data
'0x', // salt
new BN(1000 * CENTS) // deposit
);
contractId = await new Promise((resolve, reject) => {
deployTx.signAndSend(alice, ({ status, events }) => {
if (status.isInBlock) {
const contractDeployedEvent = events.find(
({ event }) => api.events.phalaComputation.ContractDeployed.is(event)
);
resolve(contractDeployedEvent.event.data.contractId);
}
});
```

```
});

// 4. SDK 컨트랙트 인스턴스 생성

contract = new Phala.PhatContract(clusterId, contractId, contractAbi);

});

it('should query contract state', async function () {

// 컨트랙트 상태 쿼리

const result = await contract.query('get');

assert.isDefined(result);

});

it('should execute contract transaction', async function () {

// 컨트랙트 트랜잭션 실행

const result = await contract.tx('flip', [], { account: alice });

assert.isDefined(result.txHash);

// 상태 변경 확인

await sleep(blockInterval * 2);

const newState = await contract.query('get');

// 상태가 변경되었는지 확인

});

});
```

5. 테스트 유틸리티

5.1 헬퍼 함수

```
// 테스트 유틸리티 함수 (추정)
const utils = {
// 조건이 만족될 때까지 대기
async checkUntil(condition, timeout = 30000, interval = 1000) {
const start = Date.now();
while (Date.now() - start < timeout) {
if (await condition()) {
return true;
}
await this.sleep(interval);
}
throw new Error(`Condition not met within ${timeout}ms`);
},
// 지정된 시간만큼 대기
sleep(ms) {
return new Promise(resolve => setTimeout(resolve, ms));
},
// 느린 테스트 스킵
skipSlowTest() {
if (process.env.SKIP_SLOW_TESTS === '1') {
return true;
}
```

```
return false;
},
// 랜덤 포트 찾기
async findAvailablePort(startPort = 3000) {
return await portfinder.getPortPromise({ port: startPort });
},
// 블록 대기
async waitForBlocks(api, blockCount) {
const startBlock = await api.rpc.chain.getBlock();
const targetBlock = startBlock.block.header.number.toNumber() + blockCount;
return new Promise((resolve) => {
const unsubscribe = api.rpc.chain.subscribeNewHeads((header) => {
if (header.number.toNumber() >= targetBlock) {
unsubscribe();
resolve(header);
}
});
});
}
};
```

5.2 Mock 데이터 생성

```
// Mock 데이터 생성 유틸리티 (추정)
const mockUtils = {
// Mock SGX Attestation 생성
generateMockAttestation(workerPubkey) {
const mockPayload = {
version: 4,
sign_type: 1,
epid_group_id: new Uint8Array(4),
qe_svn: 1,
pce_svn: 1,
xeid: 0,
basename: new Uint8Array(32),
report_body: {
cpu_svn: new Uint8Array(16),
misc_select: 0,
attributes: new Uint8Array(16),
mr_enclave: new Uint8Array(32),
mr_signer: new Uint8Array(32),
isv_prod_id: 0,
isv_svn: 0,
report_data: workerPubkey
}
```

```
};

// Mock 서명 생성 (실제로는 SGX에서 생성)

const signature = new Uint8Array(64);

crypto.getRandomValues(signature);

return {

payload: JSON.stringify(mockPayload),

signature: Array.from(signature)

};

},

// Mock 컨트랙트 ABI 생성

generateMockContractAbi() {

return {

source: {

hash: "0x1234567890abcdef",

language: "ink! 4.0.0",

compiler: "rustc 1.70.0"

},

contract: {

name: "flipper",

version: "0.1.0",

authors: ["Phala Network"]

},

spec: {
```

```
constructors: [
{
args: [
{
label: "init_value",
type: { displayName: ["bool"], type: 1 }
}
],
docs: [],
label: "new",
payable: false,
selector: "0x9bae9d5e"
}
],
docs: [],
events: [],
messages: [
{
args: [],
docs: [],
label: "flip",
mutates: true,
payable: false,
```

```
returnType: null,
selector: "0x633aa551"
},
{
args: [],
docs: [],
label: "get",
mutates: false,
payable: false,
returnType: { displayName: ["bool"], type: 1 },
selector: "0x2f865bd9"
}
]
},
types: [
{
id: 1,
type: {
def: { primitive: "bool" },
path: ["bool"]
}
}
]
```

```
};

}

};
```

6. CI/CD 통합

6.1 GitHub Actions 설정 (추정)

```yaml
# .github/workflows/e2e-test.yml

name: E2E Tests


on:

push:

branches: [ master, develop ]

pull_request:

branches: [ master ]


jobs:

e2e-test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

with:
```

```yaml
submodules: recursive

- name: Setup Rust

uses: actions-rs/toolchain@v1

with:

toolchain: stable

target: wasm32-unknown-unknown

- name: Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '18'

cache: 'yarn'

cache-dependency-path: 'e2e/yarn.lock'

- name: Build Phala Node

run: |

cargo build --release --bin phala-node

- name: Build pRuntime

run: |

cd standalone/pruntime

make

- name: Build Pherry

run: |

cargo build --release --bin pherry

- name: Install E2E Dependencies
```

```
run: |
cd e2e
yarn install
- name: Run E2E Tests
run: |
cd e2e
yarn test
env:
E2E_IN_SGX: '0'
QUICK: '1'
```

6.2 테스트 리포팅

```
// 테스트 결과 리포팅 (추정)
const testReporter = {
generateReport(results) {
const report = {
timestamp: new Date().toISOString(),
environment: {
sgxMode: inSgx,
quickMode: quickMode,
nodeVersion: process.version
},
```

```
summary: {
total: results.length,
passed: results.filter(r => r.status === 'passed').length,
failed: results.filter(r => r.status === 'failed').length,
skipped: results.filter(r => r.status === 'skipped').length
},
details: results
};
// JSON 리포트 저장
fs.writeFileSync('test-report.json', JSON.stringify(report, null, 2));
// HTML 리포트 생성
this.generateHtmlReport(report);
return report;
},
generateHtmlReport(report) {
const html = `
<!DOCTYPE html>
<html>
<head>
<title>Phala E2E Test Report</title>
<style>
body { font-family: Arial, sans-serif; margin: 20px; }
.summary { background: #f5f5f5; padding: 15px; border-radius: 5px; }
```

```
.passed { color: green; }

.failed { color: red; }

.skipped { color: orange; }

</style>

</head>

<body>

<h1>Phala E2E Test Report</h1>

<div class="summary">

<h2>Summary</h2>

<p>Total: ${report.summary.total}</p>

<p class="passed">Passed: ${report.summary.passed}</p>

<p class="failed">Failed: ${report.summary.failed}</p>

<p class="skipped">Skipped: ${report.summary.skipped}</p>

</div>

<!-- 상세 결과 표시 -->

</body>

</html>

`;

fs.writeFileSync('test-report.html',  html);

}

};
```

이 문서는 Phala Network의 E2E 테스트 시스템을 다룹니다. 다음 문서에서는 개발 도구와 스크립트들을 분석합니다.